

## ✓ Required Library Install

```
!pip install scikit-learn
```

```
!pip install tensorflow
```

```
!pip install matplotlib
```

```
!pip install plotly
```

```
!pip install seaborn
```

```
Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from seaborn) (3.10.1)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.1.0)
Requirement already satisfied: cycler>=0.10 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.22.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.1)
Requirement already satisfied: packaging>=20.0 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.0)
Requirement already satisfied: pillow>=8 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from pandas>=1.2->seaborn) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from pandas>=1.2->seaborn) (2024.1)
Requirement already satisfied: six>=1.5 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)
Installing collected packages: seaborn
Successfully installed seaborn-0.13.2
```

```
!pip install imblearn
```

```
Collecting imblearn
  Downloading imblearn-0.0-py2.py3-none-any.whl.metadata (355 bytes)
Collecting imbalanced-learn (from imblearn)
  Downloading imbalanced_learn-0.13.0-py3-none-any.whl.metadata (8.8 kB)
Requirement already satisfied: numpy<3,>=1.24.3 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from imbalanced-learn->imblearn) (1.26.4)
Requirement already satisfied: scipy<2,>=1.10.1 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from imbalanced-learn->imblearn) (1.11.0)
Requirement already satisfied: scikit-learn<2,>=1.3.2 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from imbalanced-learn->imblearn) (1.3.2)
Collecting sklearn-compat<1,>=0.1 (from imbalanced-learn->imblearn)
  Downloading sklearn_compat-0.1.3-py3-none-any.whl.metadata (18 kB)
Requirement already satisfied: joblib<2,>=1.1.1 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from sklearn-compat<1,>=0.1->imbalanced-learn->imblearn) (1.3.2)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in c:\users\nikky\anaconda3\envs\rakaenv\lib\site-packages (from sklearn-compat<1,>=0.1->imbalanced-learn->imblearn) (3.2.0)
Downloading imblearn-0.0-py2.py3-none-any.whl (1.9 kB)
Downloading imbalanced_learn-0.13.0-py3-none-any.whl (238 kB)
Downloading sklearn_compat-0.1.3-py3-none-any.whl (18 kB)
Installing collected packages: sklearn-compat, imbalanced-learn, imblearn
Successfully installed imbalanced-learn-0.13.0 imblearn-0.0 sklearn-compat-0.1.3
```

```
!pip install xgboost
```

## ✓ Required Library Import

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np
```

```
from collections import Counter as C
```

```
import plotly.graph_objects as go

import plotly.subplots as sp
from scipy.stats import gaussian_kde

import seaborn as sns
import matplotlib.pyplot as plt

from imblearn.over_sampling import SMOTE

import plotly.graph_objects as go
import plotly.express as px

from sklearn.metrics import precision_score, recall_score, f1_score, roc_auc_score, accuracy_score
from sklearn.metrics import classification_report, roc_auc_score, confusion_matrix

from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

from plotly.subplots import make_subplots

import joblib
```

## ✓ Load the Dataset

```
# df = pd.read_excel("assignment.xlsx", sheet_name="assignment", engine='openpyxl')
df = pd.read_csv("assignment.csv")
```

## ✓ Understanding Data

**Goal:** Understand the structure and size of the dataset.

- Check the number of rows and columns.
- Examine column names and data types.
- Identify missing values and duplicate rows.

```
# Check the number of rows and columns
print("Shape of the dataset:", df.shape)
print("-"*100)

# Examine column names and data types
print("\nColumn names and data types:")
print(df.dtypes)
print("-"*100)

# Identify missing values
print("\nMissing values per column:")
print(df.isnull().sum())
print("-"*100)

# Check for duplicate rows
duplicate_rows = df.duplicated().sum()
print(f"\nNumber of duplicate rows: {duplicate_rows}")

print("-"*100)

# Check for duplicates
print(df.duplicated().sum())
```

↗ Shape of the dataset: (417133, 8)

```
-----
Column names and data types:
0.027364786518591047    float64
0.9206688011383849     float64
0.6546336206896551     float64
0.15422885572139303    float64
0.1044776119402985     float64
0.5573770491803279     float64
```

```
0.75          float64
0.0           float64
dtype: object
-----

Missing values per column:
0.027364786518591047    0
0.9206688011383849      0
0.6546336206896551      0
0.15422885572139303     0
0.1044776119402985      0
0.5573770491803279      0
0.75                    0
0.0                     0
dtype: int64
-----

Number of duplicate rows: 366166
-----

366166
```

```
# View the first 5 rows of the dataset
print("First 5 rows:")
df.head()
```

↗ First 5 rows:

|   | 0.027364786518591047 | 0.9206688011383849 | 0.6546336206896551 | 0.15422885572139303 | 0.1044776119402985 | 0.5573770491803279 | 0.75 | 0.0 |
|---|----------------------|--------------------|--------------------|---------------------|--------------------|--------------------|------|-----|
| 0 | 0.027365             | 0.920669           | 0.654634           | 0.154229            | 0.104478           | 0.557377           | 0.75 | 0.0 |
| 1 | 0.027365             | 0.920669           | 0.654634           | 0.154229            | 0.104478           | 0.557377           | 0.75 | 0.0 |
| 2 | 0.027365             | 0.920669           | 0.654634           | 0.154229            | 0.104478           | 0.557377           | 0.75 | 0.0 |
| 3 | 0.026695             | 0.920669           | 0.654634           | 0.154229            | 0.104478           | 0.360656           | 0.75 | 0.0 |
| 4 | 0.026695             | 0.920669           | 0.654634           | 0.154229            | 0.104478           | 0.360656           | 0.75 | 0.0 |

```
# View the last 5 rows of the dataset
print("\nLast 5 rows:")
df.tail()
```

↗ Last 5 rows:

|        | 0.027364786518591047 | 0.9206688011383849 | 0.6546336206896551 | 0.15422885572139303 | 0.1044776119402985 | 0.5573770491803279 | 0.75 |
|--------|----------------------|--------------------|--------------------|---------------------|--------------------|--------------------|------|
| 417128 | 0.0                  | 0.972252           | 0.705819           | 0.154229            | 0.107463           | 0.606557           | 0.50 |
| 417129 | 0.0                  | 0.998577           | 0.983836           | 0.154229            | 0.102488           | 0.295082           | 0.75 |
| 417130 | 0.0                  | 0.998577           | 0.983836           | 0.154229            | 0.102488           | 0.295082           | 0.75 |
| 417131 | 0.0                  | 0.998577           | 0.983836           | 0.154229            | 0.102488           | 0.295082           | 0.75 |
| 417132 | 0.0                  | 0.998577           | 0.983836           | 0.154229            | 0.102488           | 0.295082           | 0.75 |

```
# Set columns name
df.columns = ['feature1', 'feature2', 'feature3', 'feature4', 'feature5', 'feature6', 'feature7', 'target_cols']
```

```
# View the first 5 rows of the dataset
print("First 5 rows:")
df.head()
```

↗ First 5 rows:

|   | feature1 | feature2 | feature3 | feature4 | feature5 | feature6 | feature7 | target_cols |
|---|----------|----------|----------|----------|----------|----------|----------|-------------|
| 0 | 0.027365 | 0.920669 | 0.654634 | 0.154229 | 0.104478 | 0.557377 | 0.75     | 0.0         |
| 1 | 0.027365 | 0.920669 | 0.654634 | 0.154229 | 0.104478 | 0.557377 | 0.75     | 0.0         |
| 2 | 0.027365 | 0.920669 | 0.654634 | 0.154229 | 0.104478 | 0.557377 | 0.75     | 0.0         |
| 3 | 0.026695 | 0.920669 | 0.654634 | 0.154229 | 0.104478 | 0.360656 | 0.75     | 0.0         |
| 4 | 0.026695 | 0.920669 | 0.654634 | 0.154229 | 0.104478 | 0.360656 | 0.75     | 0.0         |

```
# Summary statistics for all numerical features
summary_stats = df.describe().T # Transpose for better readability
summary_stats['median'] = df.median() # Add median
summary_stats['missing_values'] = df.isnull().sum()
summary_stats['skewness'] = df.skew()

# Sort by skewness to detect highly skewed features
summary_stats_sorted = summary_stats.sort_values(by='skewness', ascending=False)

# Display summary
summary_stats_sorted[['mean', 'median', 'std', 'min', '25%', '50%', '75%', 'max', 'skewness', 'missing_values']]
```

|                    | mean     | median   | std      | min | 25%      | 50%      | 75%      | max | skewness  | missing_values |
|--------------------|----------|----------|----------|-----|----------|----------|----------|-----|-----------|----------------|
| <b>feature1</b>    | 0.016367 | 0.000000 | 0.126351 | 0.0 | 0.000000 | 0.000000 | 0.000000 | 1.0 | 7.602783  | 0              |
| <b>feature4</b>    | 0.167478 | 0.154229 | 0.106846 | 0.0 | 0.154229 | 0.154229 | 0.154229 | 1.0 | 6.809413  | 0              |
| <b>feature5</b>    | 0.122958 | 0.104478 | 0.125154 | 0.0 | 0.102488 | 0.104478 | 0.104478 | 1.0 | 6.499746  | 0              |
| <b>target_cols</b> | 0.334351 | 0.000000 | 0.471764 | 0.0 | 0.000000 | 0.000000 | 1.000000 | 1.0 | 0.702256  | 0              |
| <b>feature6</b>    | 0.408283 | 0.418033 | 0.180911 | 0.0 | 0.295082 | 0.418033 | 0.549180 | 1.0 | -0.270317 | 0              |
| <b>feature7</b>    | 0.636488 | 0.750000 | 0.127767 | 0.0 | 0.500000 | 0.750000 | 0.750000 | 1.0 | -0.335595 | 0              |
| <b>feature3</b>    | 0.635395 | 0.674030 | 0.201498 | 0.0 | 0.591595 | 0.674030 | 0.742457 | 1.0 | -1.236799 | 0              |
| <b>feature2</b>    | 0.834862 | 0.932053 | 0.220996 | 0.0 | 0.912131 | 0.932053 | 0.967983 | 1.0 | -1.513799 | 0              |

## ✓ Target Variable Analysis : target\_cols

Investigate the nature of the target variable.

Determine if the dataset is imbalanced.

Understand the distribution of classes (e.g., 0 vs. 1).

Visualize class frequencies.

In binary classification tasks where class imbalance can significantly affect model performance.

```
print(df['target_cols'].value_counts())
print("#"*100)
print(df['target_cols'].value_counts(normalize=True) * 100)
```

```
target_cols
0.0    277664
1.0    139469
Name: count, dtype: int64
#####
target_cols
0.0     66.564861
1.0     33.435139
Name: proportion, dtype: float64
```

```
def plot_binary_target_distribution_plotly(data, target_col):
    """
    Plots the distribution of a binary target column using a Plotly bar chart.

    Parameters:
    - data (pd.DataFrame): The DataFrame containing the target column.
    - target_col (str): The name of the binary target column.
    """
    # Count values
    value_counts = data[target_col].value_counts().sort_index()
    labels = value_counts.index.astype(str)
    counts = value_counts.values

    # Create bar chart
    fig = go.Figure(data=[
        go.Bar(x=labels, y=counts, text=counts, textposition='auto', marker_color=['skyblue', 'salmon'])
    ])
    fig.show()
```

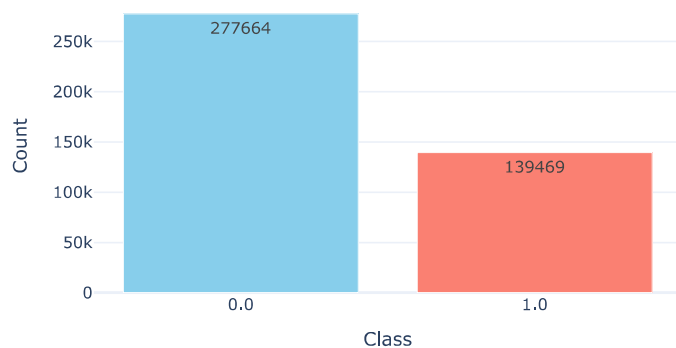
```
# Update layout with size
fig.update_layout(
    title=f'Distribution of Binary Target Column: {target_col}',
    xaxis_title='Class',
    yaxis_title='Count',
    template='plotly_white',
    width=600, # Width in pixels
    height=400 # Height in pixels
)

fig.show()
```

```
plot_binary_target_distribution_plotly(df, 'target_cols')
```



Distribution of Binary Target Column: target\_cols



#### ▼ Note : Dataset is imbalance

target\_cols : total of records

0.0 277664

1.0 139469

#####

total no of records in percentage

0.0 : 66.564861 %

1.0 : 33.435139 %

Start coding or [generate](#) with AI.

#### ▼ Feature Distributions Analysis

Visualize how individual features are distributed.

Use histograms and KDE plots.

Analyze how each feature behaves across different target classes.

understanding skewness, spread, and modality of each feature.

```
def plot_feature_histograms_plotly(data, bins=30, width=1200, height=1000):
    """
    Plots histograms for each numeric feature in a DataFrame using Plotly subplots.

    Parameters:
    - data (pd.DataFrame): The input DataFrame.
    - bins (int): Number of histogram bins.
```

```
- width (int): Width of the overall figure.
- height (int): Height of the overall figure.
"""

numeric_cols = data.select_dtypes(include='number').columns
num_features = len(numeric_cols)
cols = 3 # number of subplot columns
rows = -(num_features // cols) # ceiling division for rows

fig = sp.make_subplots(rows=rows, cols=cols, subplot_titles=numeric_cols)

for i, col in enumerate(numeric_cols):
    row = i // cols + 1
    col_pos = i % cols + 1
    fig.add_trace(
        go.Histogram(x=data[col], nbinsx=bins, name=col),
        row=row, col=col_pos
    )

fig.update_layout(
    title="Histograms for Each Feature",
    showlegend=False,
    height=height,
    width=width,
    template='plotly_white'
)
fig.show()

df_features = df.iloc[:, :-1]
plot_feature_histograms_plotly(df_features)
```



## Histograms for Each Feature



```
def plot_kde_by_class_plotly(data, target_col, width=900, height=400):
    """
    Plots KDE-like distributions for each feature by target class using Plotly.

    Parameters:
    - data (pd.DataFrame): DataFrame containing features and target column.
    - target_col (str): Name of the binary/class target column.
    - width (int): Width of the overall figure
    - height (int): Height of the overall figure
    """
    feature_cols = data.columns.difference([target_col])

    for col in feature_cols:
        # Create an empty figure
        fig = go.Figure()

        # For each target class, compute the KDE
        for target in data[target_col].unique():
            class_data = data[data[target_col] == target][col]
            kde = gaussian_kde(class_data)
```

```
x_values = np.linspace(class_data.min(), class_data.max(), 100)
y_values = kde(x_values)

fig.add_trace(go.Scatter(
    x=x_values,
    y=y_values,
    mode='lines',
    name=f'Class {target}',
    fill='tozeroy' if target == data[target_col].min() else None
))
```

```
# Update layout with custom width and height
fig.update_layout(
    title=f'Distribution of {col} by Target Class',
    xaxis_title=col,
    yaxis_title='Density',
    template='plotly_white',
    showlegend=True,
    width=width,      # Set the width
    height=height     # Set the height
)
```

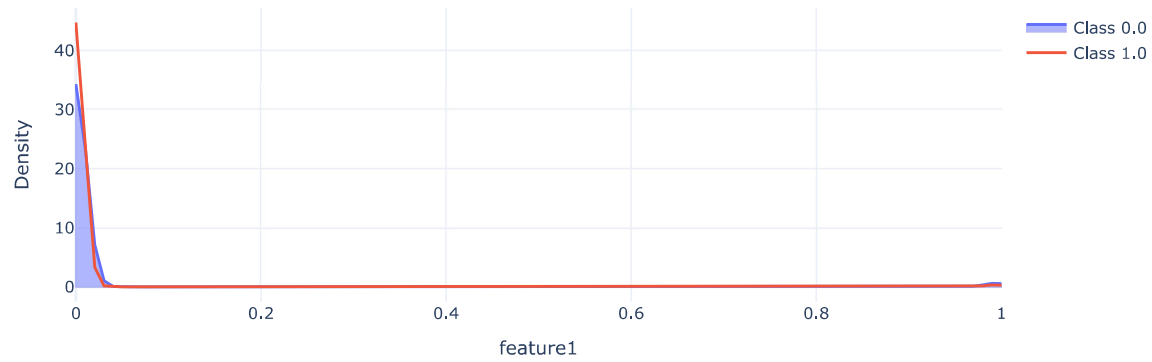
```
# Show the figure
fig.show()
```

```
plot_kde_by_class_plotly(df, target_col='target_cols')
```

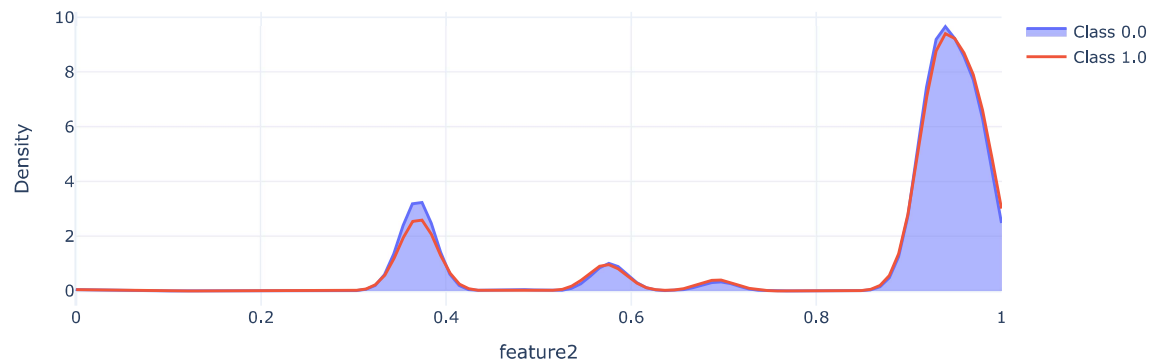




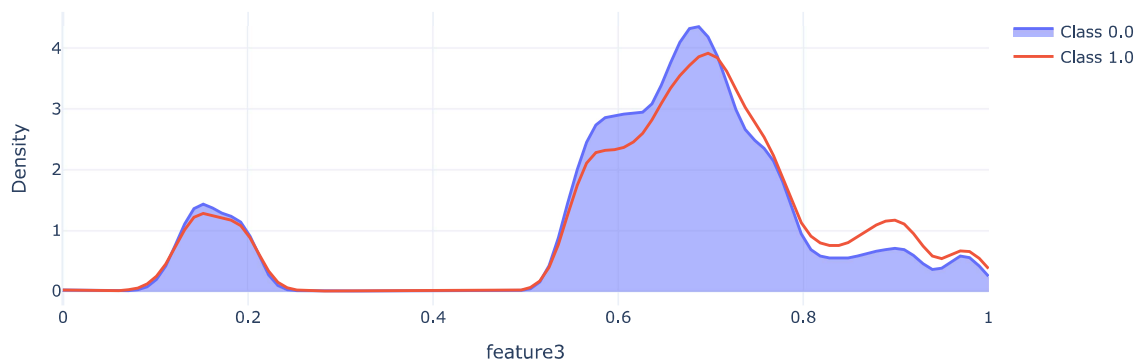
Distribution of feature1 by Target Class



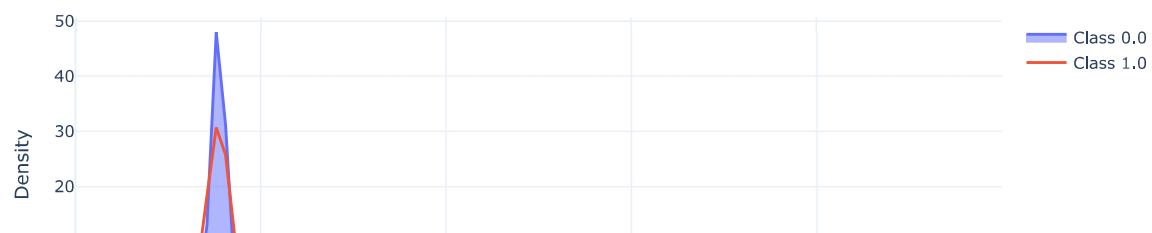
Distribution of feature2 by Target Class

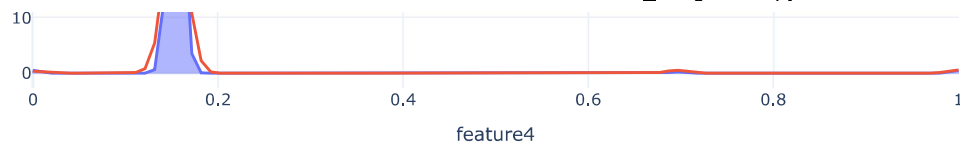


Distribution of feature3 by Target Class

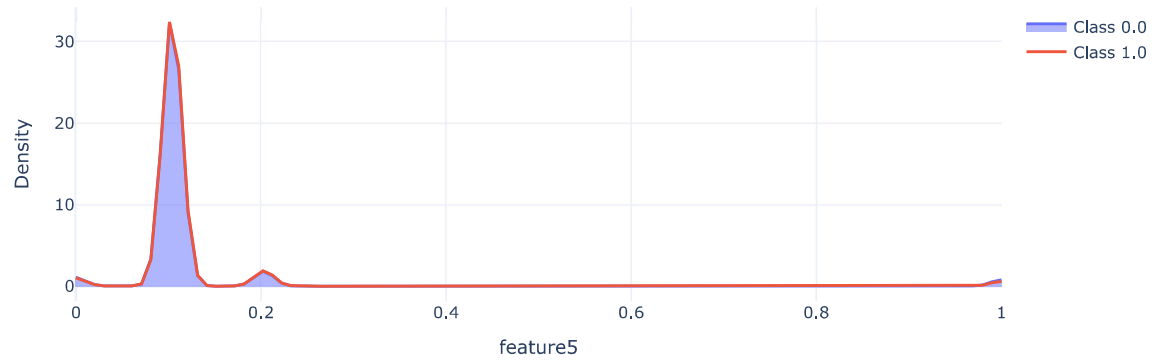


Distribution of feature4 by Target Class

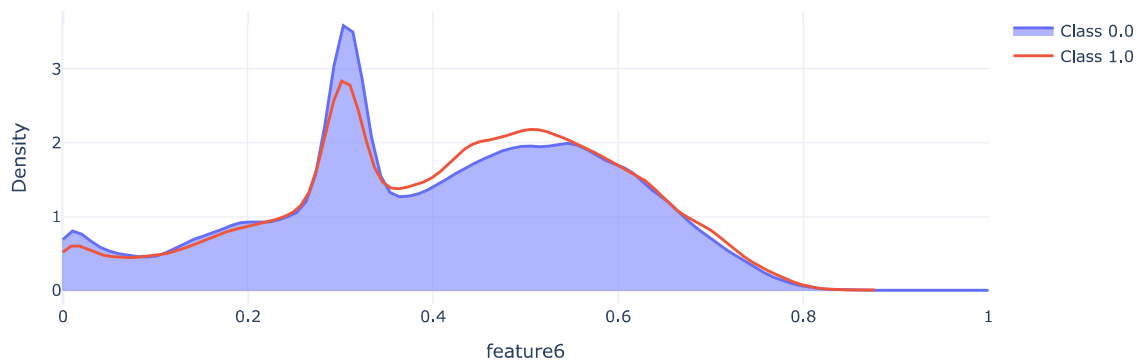




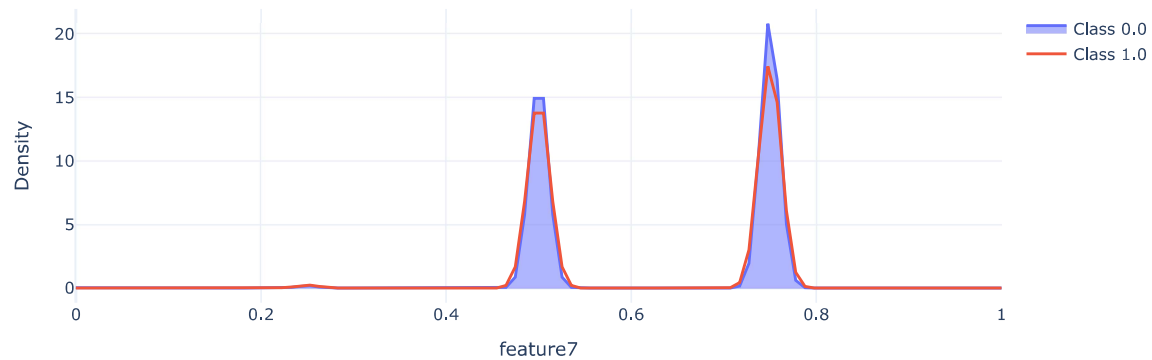
Distribution of feature5 by Target Class



Distribution of feature6 by Target Class



Distribution of feature7 by Target Class



Start coding or [generate](#) with AI.

## ✓ Correlation Analysis

Identify linear relationships between features.

Use a heatmap to spot multicollinearity.

Check feature-to-target correlation to find useful predictors.

```
def plot_correlation_matrix_plotly(data, target_col):
    # Compute the correlation matrix
    corr_matrix = data.corr()

    # Create the heatmap for the correlation matrix
    fig = go.Figure(data=go.Heatmap(
        z=corr_matrix.values,
        x=corr_matrix.columns,
        y=corr_matrix.index,
        colorscale='RdBu', # Updated to a valid colorscale
        zmin=-1, zmax=1, # Scale from -1 to 1
        colorbar=dict(title="Correlation", tickvals=[-1, 0, 1]),
        text=corr_matrix.values, # Show the correlation values
        hovertemplate='%{text}'
    ))

    fig.update_layout(
        title="Feature Correlation Matrix",
        xaxis_title="Features",
        yaxis_title="Features",
        template='plotly_white',
        width=800,
        height=600
    )
    fig.show()

    # Correlation of features with the target column
    target_corr = corr_matrix[target_col].sort_values(ascending=False)

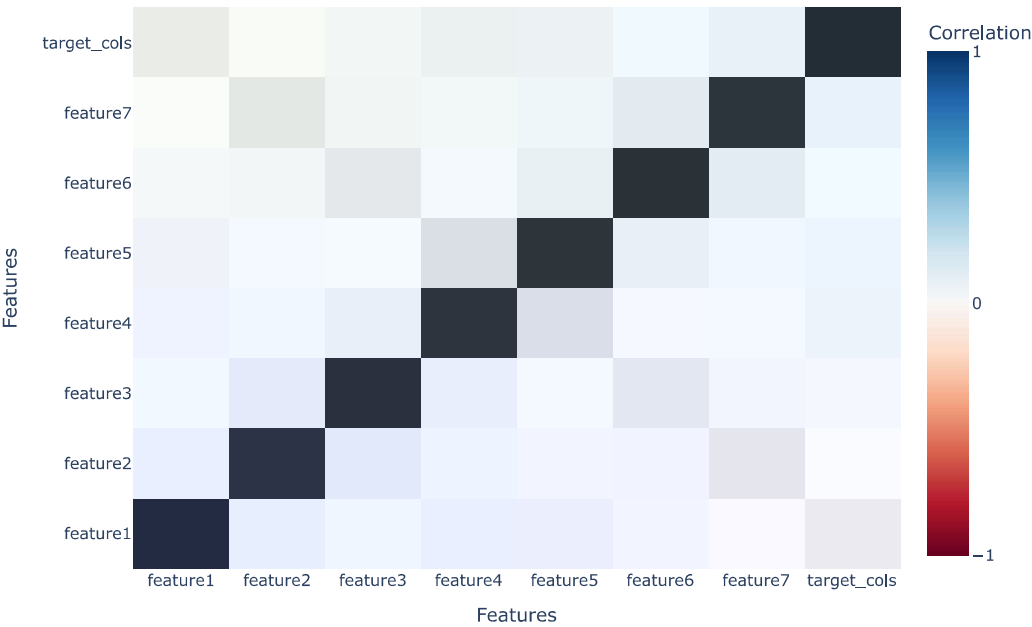
    # Plot the correlation of features with the target column
    bar_fig = px.bar(
        target_corr,
        x=target_corr.index,
        y=target_corr.values,
        labels={'x': 'Features', 'y': 'Correlation with ' + target_col},
        title=f'Correlation of Features with {target_col}',
        color=target_corr.values,
        color_continuous_scale='Viridis' # Valid continuous color scale for the bars
    )
    bar_fig.update_layout(
        template='plotly_white',
        width=800,
        height=500
    )
    bar_fig.show()

    # Optionally print correlation of features with target
    print(target_corr)

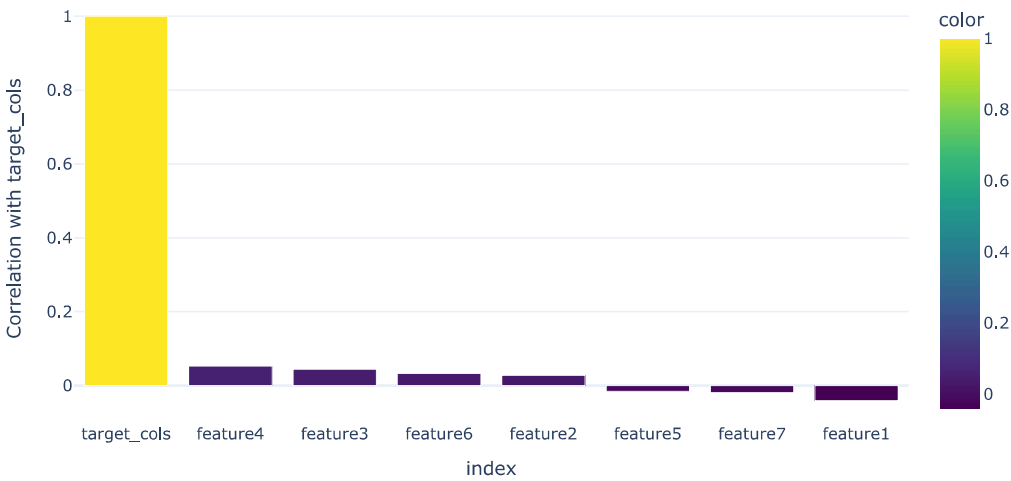
plot_correlation_matrix_plotly(df, target_col='target_cols')
```



Feature Correlation Matrix



Correlation of Features with target\_cols



```
target_cols    1.000000
feature4       0.052604
feature3       0.044232
feature6       0.033055
feature2       0.027771
feature5      -0.015940
feature7      -0.019434
feature1      -0.040850
Name: target_cols, dtype: float64
```

Start coding or [generate](#) with AI.

## Outlier Detection

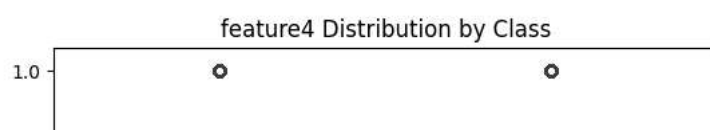
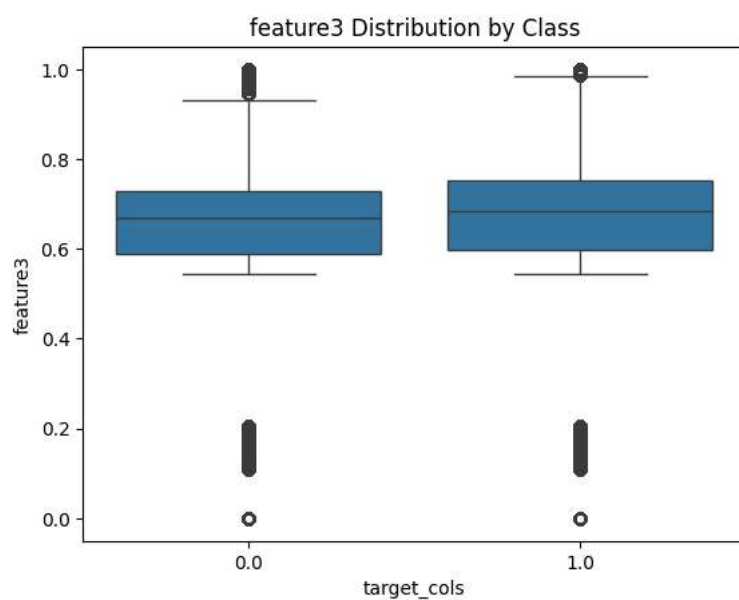
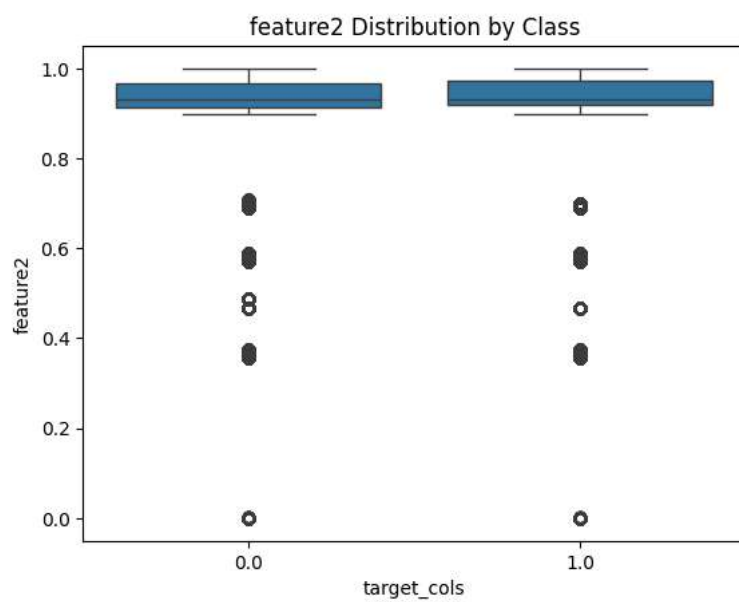
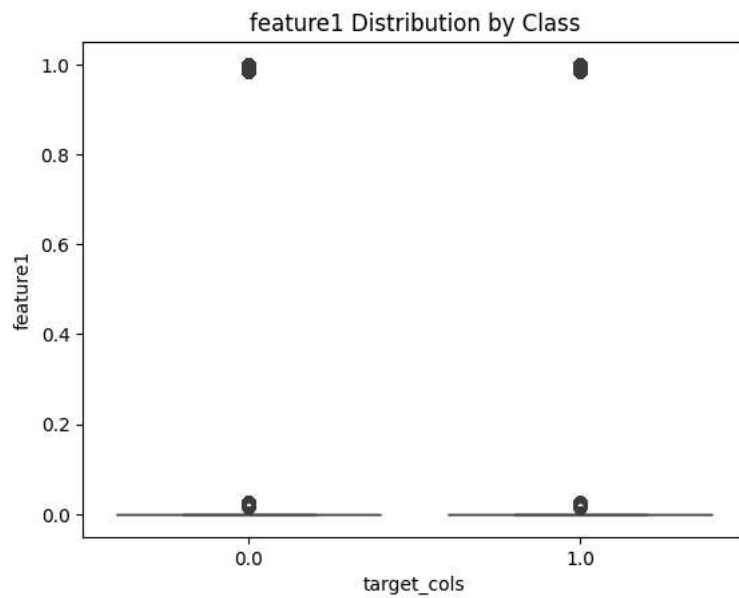
Identify extreme values that may distort the model.

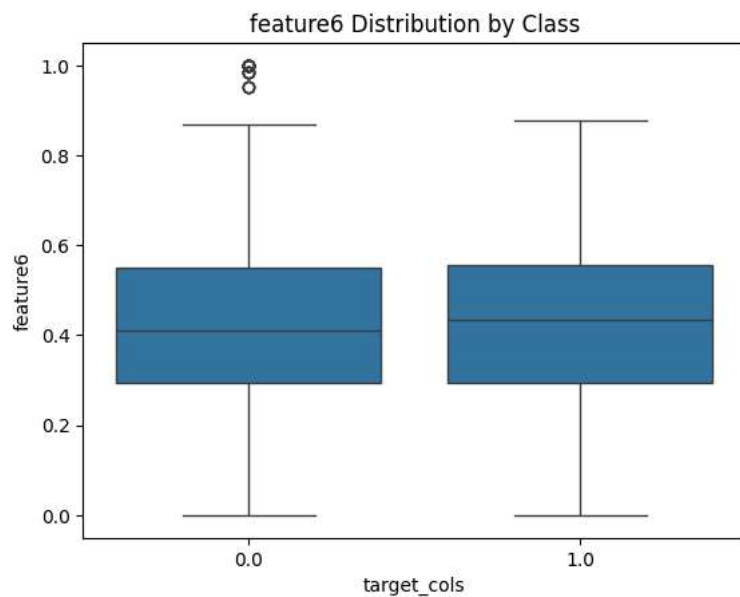
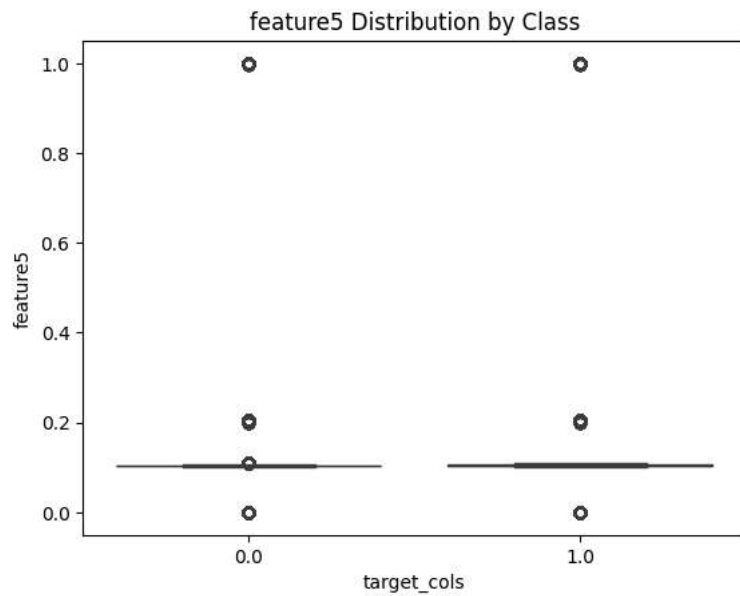
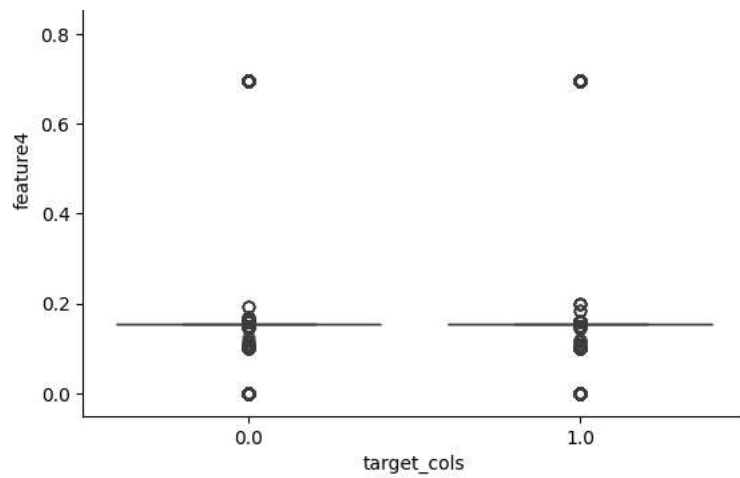
Use boxplots for each feature.

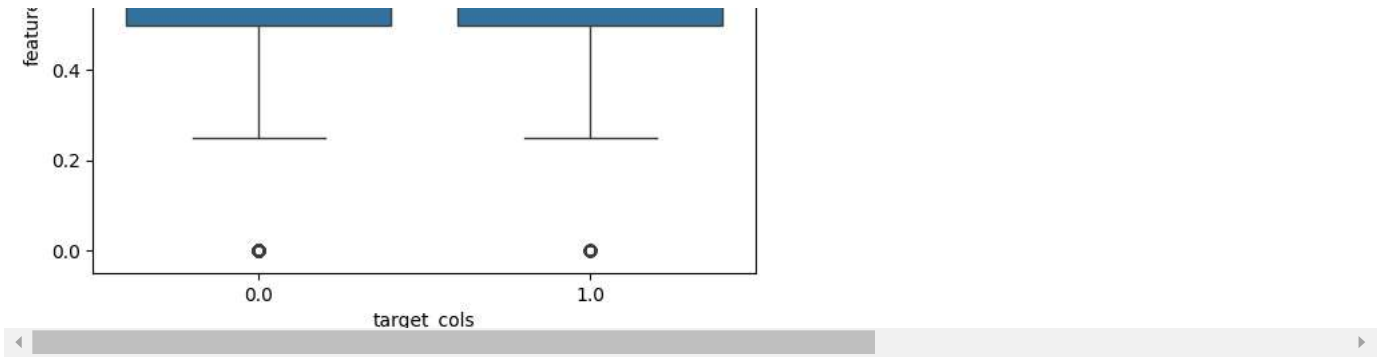
Helps determine if outlier handling (removal or transformation) is necessary.

Start coding or [generate](#) with AI.

```
for col in df.columns[:-1]:
    sns.boxplot(x='target_cols', y=col, data=df)
    plt.title(f'{col} Distribution by Class')
    plt.show()
```









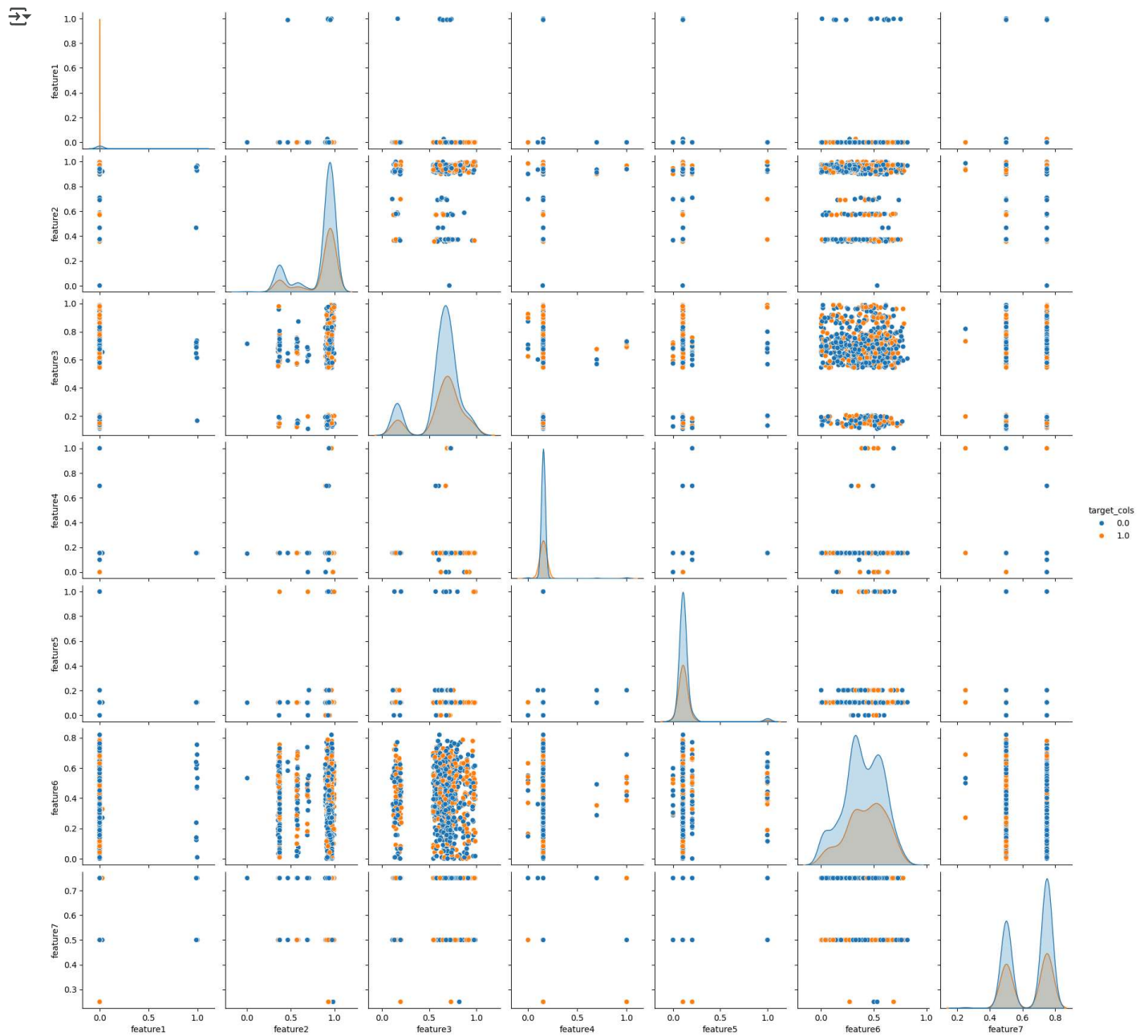
Start coding or [generate](#) with AI.

## ✓ Feature Relationships

Goal: Understand interactions between multiple features.

Use `pairplot()` to see feature combinations and their separability across target classes.

```
sns.pairplot(df.sample(1000), hue='target_cols')  
plt.show()
```



# <font color='Green'>Feature and Target Separation</font>

## ✓ Feature and Target Selection

```
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values
```

```
#checking Imbalance
print(C(y))
```

```
↗ Counter({0.0: 277664, 1.0: 139469})
```

## ✓ Handling Imbalanced Datasets Problem

```
print(df['target_cols'].value_counts())
print("#"*100)
print(df['target_cols'].value_counts(normalize=True) * 100)
```

```
↗ target_cols
0.0    277664
1.0    139469
Name: count, dtype: int64
#####
target_cols
0.0    66.564861
1.0    33.435139
Name: proportion, dtype: float64
```

```
# Handle class imbalance using SMOTE
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)
```

```
#checking balance
print(C(y_resampled))
print(len(X_resampled))
```

```
↗ Counter({0.0: 277664, 1.0: 277664})
555328
```

## ✓ Train-Test Split

dataset into training and testing subsets. use 80% for training and 20% for testing

```
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, test_size=0.2, random_state=42)
```

## ✓ Feature Scaling

Always fit the scaler on the training set only and then apply it to the test set. This prevents data leakage and ensures proper generalization.

```
# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Determine the number of unique target classes
num_classes = len(np.unique(y_train))
print(num_classes)
```

```
↗ 2
```

## ✓ Define the Embedding Model

Deep neural network that reduces the high-dimensional feature input into a 2D embedding.

```
# Define the embedding model
embedding_dim = 2

# Define the input shape based on number of features
input_shape = (X_train_scaled.shape[1],)

embedding_model = keras.Sequential([
    layers.Input(shape=input_shape), # Input layer

    # First hidden layer without BatchNormalization and Dropout
    layers.Dense(128, activation='relu'),

    # Second hidden layer with BatchNormalization and Dropout
    layers.Dense(64, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.3),

    # Third hidden layer with BatchNormalization and Dropout
    layers.Dense(32, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.3),

    # Output layer: 2D embedding
    layers.Dense(embedding_dim)
])
```

Start coding or [generate](#) with AI.

## ✓ Define Classification Model

classification layer on top of the embedding model. The model uses the previously defined embedding as a feature extractor, and then a softmax output layer is used for class prediction.

```
# Define the classification model on top of the embedding model
classification_model = keras.Sequential([
    embedding_model, # Pre-trained embedding model
    layers.Dense(1, activation='sigmoid') # Classification layer
])
```

## ✓ Compile the Binary Classification Model

```
# Compile the binary classification model
classification_model.compile(
    optimizer='adam', # Adam optimizer for adaptive learning rate
    loss='binary_crossentropy', # Loss function for binary classification
    metrics=['accuracy'] # Evaluate accuracy during training
)
```

## ✓ Train the Model

epochs: The number of times the entire training dataset is passed through the model. You can increase this for better performance but risk overfitting if it's too high.

batch\_size: The number of samples processed before the model is updated. A smaller batch size might make training slower, but it can help the model generalize better.

validation\_data: The validation data (X\_test\_scaled and y\_test) allows us to monitor the model's performance on unseen data during training.

```
# Define training parameters
epochs = 40 # Number of epochs to train # used 3 epoch because it is taking time
batch_size = 32 # Batch size
```

```
# Train the classification model
history = classification_model.fit(
    X_train_scaled, y_train, # Training data
    epochs=epochs, # Number of epochs
    batch_size=batch_size, # Batch size
    validation_data=(X_test_scaled, y_test), # Validation data
    verbose=1 # Display progress
)
```

```
Epoch 1/40
13884/13884 ————— 88s 6ms/step - accuracy: 0.5418 - loss: 0.6924 - val_accuracy: 0.5600 - val_loss: 0.6819
Epoch 2/40
13884/13884 ————— 85s 6ms/step - accuracy: 0.5569 - loss: 0.6825 - val_accuracy: 0.5665 - val_loss: 0.6792
Epoch 3/40
13884/13884 ————— 132s 5ms/step - accuracy: 0.5618 - loss: 0.6811 - val_accuracy: 0.5685 - val_loss: 0.6771
Epoch 4/40
13884/13884 ————— 71s 5ms/step - accuracy: 0.5660 - loss: 0.6792 - val_accuracy: 0.5722 - val_loss: 0.6759
Epoch 5/40
13884/13884 ————— 68s 5ms/step - accuracy: 0.5690 - loss: 0.6775 - val_accuracy: 0.5736 - val_loss: 0.6753
Epoch 6/40
13884/13884 ————— 68s 5ms/step - accuracy: 0.5724 - loss: 0.6766 - val_accuracy: 0.5846 - val_loss: 0.6726
Epoch 7/40
13884/13884 ————— 69s 5ms/step - accuracy: 0.5750 - loss: 0.6753 - val_accuracy: 0.5800 - val_loss: 0.6729
Epoch 8/40
13884/13884 ————— 68s 5ms/step - accuracy: 0.5761 - loss: 0.6745 - val_accuracy: 0.5871 - val_loss: 0.6689
Epoch 9/40
13884/13884 ————— 70s 5ms/step - accuracy: 0.5783 - loss: 0.6733 - val_accuracy: 0.5904 - val_loss: 0.6683
Epoch 10/40
13884/13884 ————— 66s 5ms/step - accuracy: 0.5817 - loss: 0.6715 - val_accuracy: 0.5911 - val_loss: 0.6667
Epoch 11/40
13884/13884 ————— 68s 5ms/step - accuracy: 0.5838 - loss: 0.6705 - val_accuracy: 0.5881 - val_loss: 0.6681
Epoch 12/40
13884/13884 ————— 67s 5ms/step - accuracy: 0.5835 - loss: 0.6705 - val_accuracy: 0.5962 - val_loss: 0.6631
Epoch 13/40
13884/13884 ————— 69s 5ms/step - accuracy: 0.5873 - loss: 0.6684 - val_accuracy: 0.5930 - val_loss: 0.6639
Epoch 14/40
13884/13884 ————— 67s 5ms/step - accuracy: 0.5889 - loss: 0.6677 - val_accuracy: 0.5968 - val_loss: 0.6631
Epoch 15/40
13884/13884 ————— 70s 5ms/step - accuracy: 0.5930 - loss: 0.6667 - val_accuracy: 0.5995 - val_loss: 0.6615
Epoch 16/40
13884/13884 ————— 68s 5ms/step - accuracy: 0.5940 - loss: 0.6667 - val_accuracy: 0.6025 - val_loss: 0.6614
Epoch 17/40
13884/13884 ————— 70s 5ms/step - accuracy: 0.5955 - loss: 0.6661 - val_accuracy: 0.6022 - val_loss: 0.6612
Epoch 18/40
13884/13884 ————— 68s 5ms/step - accuracy: 0.5956 - loss: 0.6655 - val_accuracy: 0.6033 - val_loss: 0.6599
Epoch 19/40
13884/13884 ————— 69s 5ms/step - accuracy: 0.5941 - loss: 0.6660 - val_accuracy: 0.6033 - val_loss: 0.6590
Epoch 20/40
13884/13884 ————— 70s 5ms/step - accuracy: 0.5955 - loss: 0.6653 - val_accuracy: 0.6049 - val_loss: 0.6603
Epoch 21/40
13884/13884 ————— 70s 5ms/step - accuracy: 0.5970 - loss: 0.6642 - val_accuracy: 0.6071 - val_loss: 0.6596
Epoch 22/40
13884/13884 ————— 69s 5ms/step - accuracy: 0.5964 - loss: 0.6644 - val_accuracy: 0.6098 - val_loss: 0.6594
Epoch 23/40
13884/13884 ————— 70s 5ms/step - accuracy: 0.5962 - loss: 0.6644 - val_accuracy: 0.6054 - val_loss: 0.6588
Epoch 24/40
13884/13884 ————— 69s 5ms/step - accuracy: 0.5982 - loss: 0.6631 - val_accuracy: 0.5986 - val_loss: 0.6592
Epoch 25/40
13884/13884 ————— 69s 5ms/step - accuracy: 0.5978 - loss: 0.6635 - val_accuracy: 0.6062 - val_loss: 0.6563
Epoch 26/40
13884/13884 ————— 69s 5ms/step - accuracy: 0.5993 - loss: 0.6626 - val_accuracy: 0.6103 - val_loss: 0.6556
Epoch 27/40
13884/13884 ————— 71s 5ms/step - accuracy: 0.6018 - loss: 0.6617 - val_accuracy: 0.6079 - val_loss: 0.6580
Epoch 28/40
13884/13884 ————— 71s 5ms/step - accuracy: 0.6017 - loss: 0.6613 - val_accuracy: 0.6046 - val_loss: 0.6570
Epoch 29/40
13884/13884 ————— 69s 5ms/step - accuracy: 0.6026 - loss: 0.6609 - val_accuracy: 0.6045 - val_loss: 0.6562
```

## ✓ Analysis Training Progress

```
def plot_training_history_plotly(history):
    """
    Plots training and validation accuracy and loss using Plotly.

    Parameters:
    - history: Keras History object returned by model.fit()
    """
```

```
# Plot Accuracy
fig_acc = go.Figure()
fig_acc.add_trace(go.Scatter(
    y=history.history['accuracy'], mode='lines+markers', name='Train Accuracy'))
fig_acc.add_trace(go.Scatter(
    y=history.history['val_accuracy'], mode='lines+markers', name='Val Accuracy'))

fig_acc.update_layout(
    title='Model Accuracy',
    xaxis_title='Epoch',
    yaxis_title='Accuracy',
    template='plotly_white',
    width=800,
    height=500
)
fig_acc.show()

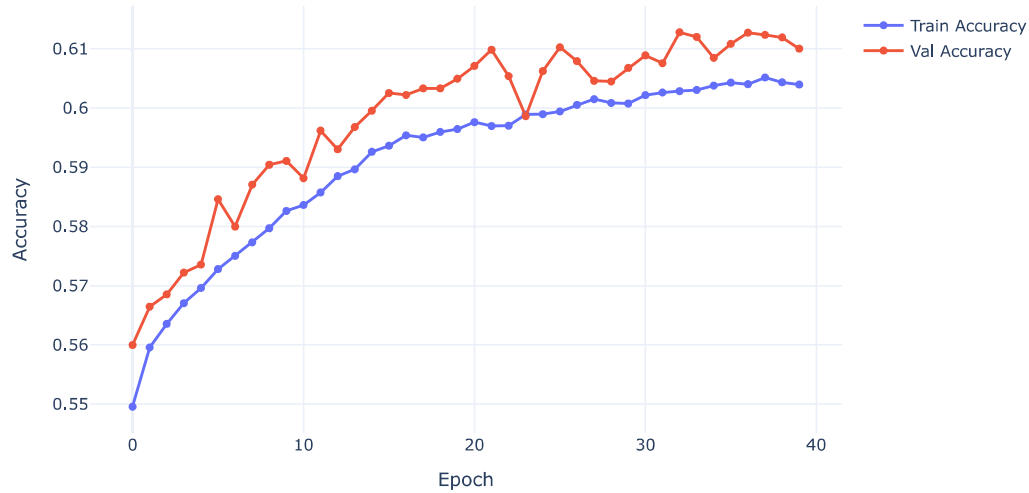
# Plot Loss
fig_loss = go.Figure()
fig_loss.add_trace(go.Scatter(
    y=history.history['loss'], mode='lines+markers', name='Train Loss'))
fig_loss.add_trace(go.Scatter(
    y=history.history['val_loss'], mode='lines+markers', name='Val Loss'))

fig_loss.update_layout(
    title='Model Loss',
    xaxis_title='Epoch',
    yaxis_title='Loss',
    template='plotly_white',
    width=800,
    height=500
)
fig_loss.show()

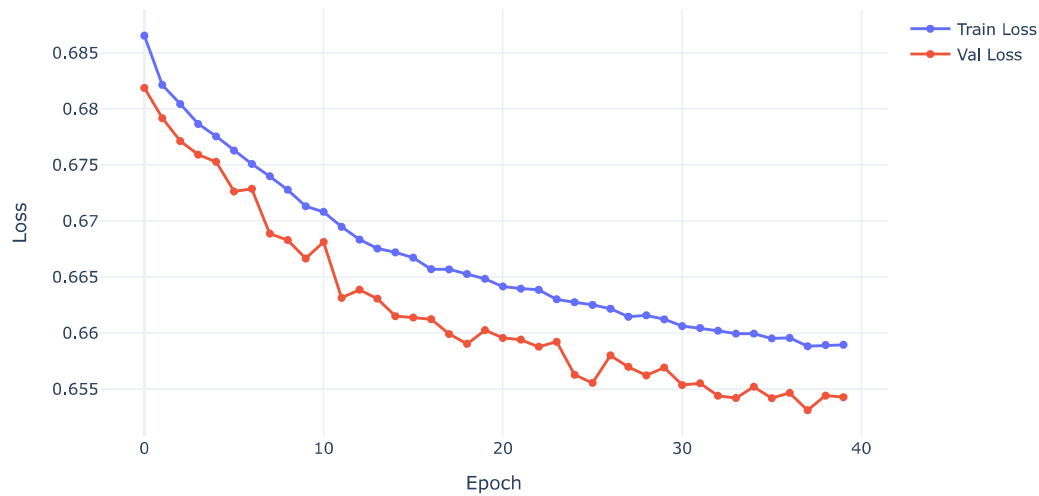
plot_training_history_plotly(history)
```



Model Accuracy



Model Loss



## ✓ Model Evaluation and Predicting Binary & Probabilities

```
loss, accuracy = classification_model.evaluate(X_test_scaled, y_test)
print(f"Test Loss: {loss:.4f}, Test Accuracy: {accuracy:.4f}")
```



3471/3471 ————— 10s 3ms/step - accuracy: 0.6097 - loss: 0.6546  
Test Loss: 0.6543, Test Accuracy: 0.6100

```
y_pred_prob = classification_model.predict(X_test_scaled).flatten()
y_pred_binary = (y_pred_prob > 0.5).astype(int)
y_pred_binary
```



3471/3471 ————— 6s 2ms/step  
array([1, 0, 0, ..., 1, 0, 0])

```
# Evaluate predictions
y_pred_prob = classification_model.predict(X_test_scaled).flatten()
y_pred_binary = (y_pred_prob > 0.5).astype(int)
```